

VISION

To build competent Engineers in the stream of Data Science to cope up with the challenges of changing industry standards by facilitating learning activities to cater the societal and personal needs.

MISSION

- To provide aspired technical education in the field of Data science by creating a learning ambience aiding towards originality and innovation.
- To establish industry academia interaction to familiarize with needs of the industry and providing exposure to the students in latest tools and technology.
- To inculcate moral ethics in students enabling them to become socially committed professionals with leadership qualities.

Program Educational Objective (PEO)

- To achieve competence in fundamentals of Data Science concepts to address the ever changing industry requirements.
- To enrich students to acquire skills needed by the industry and evolve with changing technology landscapes.

Program Specific Outcome (PSO)

- The students are able to use advanced tools and technologies which will enable them to adapt to real world projects with effective communication skills.
- The graduates will exhibit high standards of social and professional ethics in pursuing higher education and research and persevere the essence of lifelong learning in the field of Data Science Engineering.

PROGRAM OUTCOMES (PO's)

1	Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2	Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
3	Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4	Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions for complex problems: • That cannot be solved by straightforward application of knowledge, theories and techniques applicable to the engineering discipline as against problems given at the end of chapters in a typical text book that can be solved using simple engineering theories And techniques; • That may not have a unique solution .For example ,a design problem can be solved in many Ways and lead to multiple possible solutions; • That require consideration of appropriate constraints / requirements not explicitly given in the problem statement such as cost, power requirement, durability, product life, etc.; • Which need to be defined (modeled) within appropriate mathematical framework; and • That often require use of modern computational concepts and tools, for example, in the design of an antenna or a DSP filter.
5	Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6	The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7	Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9	Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11	Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12	Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Course Learning Objectives

- Provide a strong foundation in database concepts, technology, and practice.
- Practice SQL programming through a variety of database problems.
- Understand the relational database design principles.
- Demonstrate the use of concurrency and transactions in database.
- Design and build database applications for real world problems.
- Become familiar with database storage structures and access techniques.

Laboratory Outcomes

- Describe the basic elements of a relational database management system.
- Design entity relationship for the given scenario.
- Apply various Structured Query Language (SQL) statements for database manipulation.
- Analyse various normalization forms for the given application.
- Develop database applications for the given real world problem.
- Understand the concepts related to NoSQL databases.

CO/PO	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12		PSO1	PSO2
CO1	2	2	2	-	2	2	-	-	2	-	-	2		2	2
CO2	2	2	3	-	3	-	-	-	1	-	-	2		2	2
CO3	2	3	2	-	3	-	-	-	2	-	-	3		3	2
CO4	2	2	3	-	3	-	-	-	1	-	-	2		2	2
CO5	2	2	3	-	2	-	-	-	2	-	-	3		2	2
CO6	2	2	3	-	3	-	-	-	2	-	2	3		2	3

Pre Commands for setup Oracle database

1.Create a user command

Syntax: create user username identified by password;
Example: create user example identified by example;

2.Grant connect to user

Syntax:grant connect to userName identified by password;
Example:grant connect to example identified by example;

3.Grant all privileges to user

Syntax:grant all privileges to userName identified by password;
Example:grant all privileges to example identified by example;

1. Create a table called Employee & execute the following. Employee (EMPNO,ENAME,JOB, MANAGER_NO, SAL, COMMISSION)

1. Create a user and grant all permissions to the user.

2. Insert the any three records in the employee table contains attributes EMPNO, ENAME JOB, MANAGER_NO, SAL, COMMISSION and use rollback. Check the result.

3. Add primary key constraint and not null constraint to the employee table.

4. Insert null values to the employee table and verify the result.

1.Create a table called Employee & execute the following.

Employee (EMPNO, ENAME, JOB, MANAGER_NO, SAL, COMMISSION)

SOLUTION:SQL>

create table employee (empno number(10), ename varchar2(10), job varchar2(10), manager_no number(10), sal number(10), commission number(10));

SQL>

desc employee;

2.Insert the any three records in the employee table contains attributes EMPNO, ENAME JOB,MANAGER_NO, SAL, COMMISSION and use rollback. Check the result.

INSERT INTO table (column1, column2, ... column_n) VALUES (expression1, expression2, ...expression_n);

SQL>

1.insert into employee (empno, ename, job, manager_no, sal, commission)values(1,'Abhi','Manager',1,20000,1000);

2.insert into employee (empno, ename, job, manager_no, sal, commission)values(2,'Rohan','Analyst',1,10000,100);

3.insert into employee (empno, ename, job, manager_no, sal, commission)values(3,'David','Clerk',1,9000,50);

Insert command with rollback option

BEGIN insert into employee (empno, ename, job, manager_no, sal, commission)values(4,'Karan','Manager',1,20000,1000);ROLLBACK;
END;

3.Add primary key constraint and not null constraint to the employee table.

SOLUTION:To add NOT NULL constraint to existing column to the employee table.

```
ALTER TABLE tablename MODIFY columnname NOT NULL NOVALIDATE;  
SQL>
```

```
ALTER TABLE employee MODIFY empno NOT NULL NOVALIDATE;
```

To add Primary key to the employee table.

SYNTAX:

```
ALTER TABLE tablename ADD CONSTRAINT constraint name PRIMARY KEY(column1,  
column2...columnn);
```

SQL>

```
ALTER TABLE employee ADD CONSTRAINT pk_emp PRIMARY KEY (empno);
```

4.Insert null values to the employee table and verify the result.

To Insert null values, use the insert command and pass the column value as null. Only NULLABLEcolumns accept the null values.

SQL>

```
1.insert into employee (empno, ename, job, manager_no, sal,  
commission)values(5,'Kumar','Admin',NULL,NULL,NULL);
```

```
2.insert into employee (empno, ename, job, manager_no, sal, commission) values  
(6,'Suhan',NULL, NULL, NULL, NULL);
```

```
3.insert into employee (empno, ename, job, manager_no, sal, commission) values (7, NULL,  
NULL, NULL, NULL, NULL);
```

Verify the Result

SQL>

```
select * from employee;
```

2. Create a table called Employee that contain attributes EMPNO,ENAME, JOB, MGR,SAL & execute the following.

1. Add a column commission with domain to the Employee table.

2. Insert any five records into the table.

3. Update the column details of job

4. Rename the column of Employ table using alter command.

5. Delete the employee whose Empno is 105.

Pre Query:

Create a table called Employee & execute the following.

Employee (EMPNO, ENAME, JOB, MGR, SAL)

SQL>

```
create table employee (empno number(10), ename varchar2(10), job varchar2(10), mgrnumber(10), sal  
number(10));
```

SQL>

```
desc employee;
```

1.Add a column commission with domain to the Employee table.

SQL>

```
alter table employee add(commission number(10));
```

SQL>

desc employee;

2. Insert any five records into the table.

SQL>

1.insert into employee (empno, ename, job, mgr, sal, commission)values(101,'Kumar','Manager',100,20000,100);

2.insert into employee (empno, ename, job, mgr, sal, commission)values(102,'Rohan','Analyst',101,10000,100);

3.insert into employee (empno, ename, job, mgr, sal, commission)values(103,'Kumar','Clerk',101,9000,50);

4.insert into employee (empno, ename, job, mgr, sal, commission)values(104,'Chandu','Admin',101,9000,50);

5.insert into employee (empno, ename, job, mgr, sal, commission)values(105,'Keerthi','Designer',101,9000,50);

Practice Query:Check the result of insertion of rows into the employee table.

SQL> select * from employee;

3. Update the column details of job.

SOLUTION:

Syntax:

UPDATE table SET column1 = expression1, column2 = expression2,... column_n = expression_n WHERE conditions;

SQL>

update employee set job='trainee' where empno=103;

Practice Query:Check the result of updated rows in the employee table.

SQL>

select * from employee;

4. Rename the column of Employ table using alter command.

Syntax:

ALTER TABLE table_name MODIFY (column_1 column_type, column_2 column_type,... column_n column_type);

SQL>

alter table employee rename column mgr to manager_no;

Practice Query:Check the result of altered column in the employee table.

SQL>

desc employee;

5. Delete the employee whose Empno is 105.

DELETE FROM table_name WHERE conditions;

SQL>

delete employee where empno=105;

Practice Query:Check the result of deleted row in the employee table.

SQL>

select * from employee;

3. Queries using aggregate functions(COUNT,AVG,MIN,MAX,SUM),Group by, Orderby. Employee(E_id, E_name, Age, Salary)

1. Create Employee table containing all Records E_id, E_name, Age, Salary.

2. Count number of employee names from employee table

3. Find the Maximum age from employee table.

- 4. Find the Minimum age from employee table.**
- 5. Find salaries of employee in Ascending Order.**
- 6. Find grouped salaries of employees.**

1.Create a table called Employee & execute the following.

Employee (EMPNO, ENAME, AGE, SAL)

SQL>

```
create table employee (empno number(10), ename varchar2(10), age number(10), salnumber(10));
```

SQL>

```
desc employee;
```

Insert atleast five rows into the employee table.

```
1.insert into employee (empno, ename, age, sal) values(101,'Kumar',45,20000);
```

```
2.insert into employee (empno, ename, age, sal) values(102,'Rohan',42,10000);
```

```
3.insert into employee (empno, ename, age, sal) values(103,'Kumar',38,9000);
```

```
4.insert into employee (empno, ename, age, sal) values(104,'Chandu',35,9000);
```

```
5.insert into employee (empno, ename, age, sal) values(105,'Keerthi',36,9000);
```

Practice Query:Check the result of insertion of rows in the employee table.

SQL>

```
select * from employee;
```

2. Count number of employee names from employee table.

SQL>

```
select count(ename) from employee;
```

3. Find the Maximum age from employee table.

SQL>

```
select max(age) from employee;
```

4. Find the Minimum age from employee table.

SQL>

```
select min(age) from employee;
```

Practice Query:SQL>

```
select avg(age) from employee;
```

5. Find salaries of employee in Ascending Order.

SQL>

```
select ename, sal from employee order by sal;
```

Practice Query:

Find salaries of employee in DescendingOrder.

SQL>

```
select ename,sal from employee order by sal desc;
```

6. Find grouped salaries of employees.

SQL>

```
select sal from employee group by sal;
```

Practice Query:

Find employee name and salaries of employee whose age is below 40 and salary is above 5000.

SQL>

```
select ename, sal from employee where age<40 group by ename, sal having sal>5000;
```

4. Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary. CUSTOMERS(ID,NAME,AGE,ADDRESS,SALARY)

PRE QUERIES:

Create a table called customers & execute the following.

customers (ID, NAME, AGE, ADDRESS, SALARY)

SQL>

```
create table customers (id number(10), name varchar2(30), age number(10), address
varchar2(50),salary number(10));
```

Insert atleast five rows into the employee table.

- 1.insert into customers (id, name, age, address, salary) values(101,'Kumar',45, 'Bengalure',20000);
- 2.insert into customers (id, name, age, address, salary) values(102,'Rohan',42, 'Tumkuru', 10000);
- 3.insert into customers (id, name, age, address, salary) values(103,'Kumar',38, 'Belagavi', 9000);
- 4.insert into customers (id, name, age, address, salary) values(104,'Chandu',35, 'Tiptur ', 9000);
- 5.insert into customers (id, name, age, address, salary) values(105,'Keerthi',36, 'Mangaluru',9000);

Practice Query:Check the result of insertion of rows in the employee table.

SQL>

```
select * from employee;
```

1. Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old & new Salary.

SQL>

```
CREATE OR REPLACE TRIGGER salary_difference_trigger
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW
DECLARE
    v_old_salary NUMBER;
    v_new_salary NUMBER;
    v_diff NUMBER;
BEGIN
    IF INSERTING THEN
        DBMS_OUTPUT.PUT_LINE('New salary: ' || :NEW.salary);
    ELSIF UPDATING THEN
        v_old_salary := :OLD.salary;
        v_new_salary := :NEW.salary;
        v_diff := v_new_salary - v_old_salary;
        DBMS_OUTPUT.PUT_LINE('Old salary: ' || v_old_salary);
        DBMS_OUTPUT.PUT_LINE('New salary: ' || v_new_salary);
        DBMS_OUTPUT.PUT_LINE('Salary difference: ' || v_diff);
    ELSIF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('Old salary: ' || :OLD.salary);
    END IF;
END;
/
```

To check whether trigger works during INSERT statement.

SQL>

```
insert into customers (id, name, age, address, salary) values (105,'Keerthi',36, 'Mangaluru',9000);
```

To check whether trigger works during UPDATE statement.

SQL>

```
update customers SET salary=10000 where id=105;
```


To check whether trigger works during DELETE statement.

SQL>

delete from customers where id=105;

5. Create cursor for Employee table & extract the values from the table. Declare the variables ,Open the cursor & extract the values from the cursor. Close the cursor. Employee(E_id, E_name, Age, Salary)

PRE QUERIES:

Create a table called customers & execute the following.

employee (E_ID,E_NAME, AGE, SALARY)

SQL>

create table Employee (e_id number(10), e_name varchar2(30), age number(10), salary number(10));

SQL>

desc Employee;

Insert atleast five rows into the employee table.

1.insert into employee (e_id, e_name, age, salary) values(101,'Kumar',45,20000);

2.insert into employee (e_id, e_name, age, salary) values(102,'Rohan',42, 10000);

3.insert into employee (e_id, e_name, age, salary) values(103,'Kumar',38, 9000);

4.insert into employee (e_id, e_name, age, salary) values(104,'Chandu',35, 9000);

5.insert into employee (e_id, e_name, age, salary) values(105,'Keerthi',36, 9000);

Practice Query:Check the result of insertion of rows in the employee table.

SQL>

select * from Employee;

1. Create cursor for Employee table & extract the values from the table. Declare the variables, Open the cursor & extract the values from the cursor. Close the cursor.

Employee (E_id, E_name, Age, Salary)

SQL>

SET SERVEROUTPUT ON;

DECLARE

-- Declare variables to hold values fetched from the cursor

v_e_id Employee.E_id%TYPE;

v_e_name Employee.E_name%TYPE;

v_age Employee.Age%TYPE;

v_salary Employee.Salary%TYPE;

-- Declare cursor

CURSOR emp_cursor IS

SELECT E_id, E_name, Age, Salary

FROM Employee;

BEGIN

```
-- Open the cursor
OPEN emp_cursor;

-- Fetch and process each row from the cursor
LOOP
    FETCH emp_cursor INTO v_e_id, v_e_name, v_age, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Print or process fetched values (here, we'll print them)
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_e_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_e_name);
    DBMS_OUTPUT.PUT_LINE('Age: ' || v_age);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
    DBMS_OUTPUT.PUT_LINE('-----');
END LOOP;

-- Close the cursor
CLOSE emp_cursor;

END;
```

/

6. Write a PL/SQL block of code using parameterized Cursor, that will merge the data available in the newly created table N_RollCall with the data available in the table O_RollCall. If the data in the first table already exist in the second table then that data should be skipped.

PRE QUERIES:

Create a table called N_RollCall & execute the following.

N_RollCall (ROLL,NAME)

SQL>

create table N_RollCall (roll number(10), name varchar2(30));

SQL>

desc N_RollCall;

Insert atleast five rows into the N_RollCall table.

```
1.insert into N_RollCall (roll,name) values(101,'Kumar');
2.insert into N_RollCall (roll,name)values(102,'Rohan');
3.insert into N_RollCall (roll,name) values(103,'Kumar');
4.insert into N_RollCall (roll,name) values(104,'Chandu');
5.insert into N_RollCall (roll,name) values(105,'Keerthi');
```

Practice Query:Check the result of insertion of rows in the N_RollCall table.

SQL>

```
select * from N_RollCall;
```

Create a table called O_RollCall & execute the following.

O_RollCall (ROLL,NAME)

SQL>

```
create table O_RollCall (roll number(10), name varchar2(30));
```

SQL>

```
desc O_RollCall;
```

Insert atleast five rows into the O_RollCall table.

- 1.insert into O_RollCall (roll,name) values(201,'Abhi');
- 2.insert into O_RollCall (roll,name)values(202,'Rohit');
- 3.insert into O_RollCall (roll,name) values(203,'Kumar');
- 4.insert into O_RollCall (roll,name) values(204,'Chandu');
- 5.insert into O_RollCall (roll,name) values(505,'Keerthi');

Practice Query:Check the result of insertion of rows in the O_RollCall table.

SQL>

```
select * from O_RollCall;
```

1. Write a PL/SQL block of code using parameterized Cursor, that will merge the data available in the newly created table N_RollCall with the data available in the table O_RollCall. If the data in the first table already exist in the second table, then that data should be skipped.

SQL>

```
DECLARE CURSOR c1 IS SELECT * FROM O_RollCall;
roll_no number;
counter number(10);
BEGIN
counter:=0;
FOR c1_rec IN c1
LOOP select count(*) into roll_no from N_RollCall where roll=c1_rec.roll;
if roll_no=0 then INSERT INTO N_RollCall VALUES (c1_rec.roll, c1_rec.name);
counter:=counter+1;
end if;
END LOOP;
COMMIT;
DBMS_OUTPUT.PUT_LINE ('Number of Rows Inserted:'||counter);
END;
/
```

7. Install an Open Source NoSQL Data base MongoDB & perform basic CRUD(Create, Read, Update & Delete) operations. Execute MongoDB basic Queries using CRUD operations.

Step 1: Install MongoDB

1. Install MongoDB Community Edition

- Visit the official MongoDB website: [MongoDB Download Center](https://www.mongodb.com/download-center)
- Download the appropriate version for your operating system.
- Follow the installation instructions specific to your OS.

2. Start MongoDB

- After installation, start MongoDB by running the mongod command in your terminal or command prompt.

- On Windows, you might need to start MongoDB as a service or manually start it using the command line.

Step 2: Connect to MongoDB

1. Open MongoDB Shell

- Open a new terminal or command prompt.
- Run the mongo command to start the MongoDB shell and connect to the local MongoDB instance.

Step 3: Perform CRUD Operations

Now, let's perform basic CRUD operations using MongoDB shell commands.

Create Operation (Insert)

javascript

Copy code

// Switch to a specific database (e.g., 'mydatabase')

use mydatabase;

// Insert a document into a collection (e.g., 'mycollection')

db.mycollection.insertOne({

name: "John Doe",

age: 30,

city: "New York"

});

Read Operation (Find)

javascript

Copy code

// Find all documents in 'mycollection'

db.mycollection.find();

// Find documents with a specific condition (e.g., name equals "John Doe")

db.mycollection.find({ name: "John Doe" });

// Find one document based on a condition

db.mycollection.findOne({ name: "John Doe" });

Update Operation (Update)

javascript

Copy code

// Update a document (e.g., update age to 31 where name is "John Doe")

db.mycollection.updateOne(

{ name: "John Doe" },

{ \$set: { age: 31 } }

);

Delete Operation (Delete)

javascript

Copy code

// Delete a document (e.g., delete where name is "John Doe")

db.mycollection.deleteOne({ name: "John Doe" });

Additional MongoDB Queries

Here are some additional queries you can execute:

```
javascript
Copy code
// Count documents in a collection
db.mycollection.countDocuments();

// Aggregate data using aggregation pipeline (example: group by city and count)
db.mycollection.aggregate([
  { $group: { _id: "$city", count: { $sum: 1 } } }
]);

// Indexing a field (example: create index on 'name' field)
db.mycollection.createIndex({ name: 1 });
```

Exiting MongoDB Shell

To exit the MongoDB shell:

```
javascript
Copy code
// Exit MongoDB shell
quit();
```

Notes:

- Replace mydatabase and mycollection with your actual database and collection names.
- MongoDB commands are executed in JavaScript syntax within the MongoDB shell.
- Ensure MongoDB is running (mongod process) before connecting via mongo shell.

By following these steps, you can install MongoDB, connect to it, and perform basic CRUD operations as well as execute basic queries using the MongoDB shell. Adjustments can be made based on specific data models and requirements.

VIVA QUESTIONS AND ANSWERS

1.What is database?

A database is a collection of information that is organized. So that it can easily be accessed, managed, and updated.

2.What is DBMS?

DBMS stands for Database Management System. It is a collection of programs that enables user to create and maintain a database.

3.What is a Database system?

The database and DBMS software together is called as Database system.

4.What are the advantages of DBMS?

- I. Redundancy is controlled.
- II. Providing multiple user interfaces.
- III. Providing backup and recovery
- IV. Unauthorized access is restricted.
- V. Enforcing integrity constraints.

5.What is normalization?

It is a process of analysing the given relation schemas based on their Functional Dependencies (FDs) and primary key to achieve the properties (1). Minimizing redundancy, (2). Minimizing insertion, deletion and update anomalies.

6.What is Data Model?

A collection of conceptual tools for describing data, data relationships data semantics and constraints.

7.What is E-R model?

This data model is based on real world that consists of basic objects called entities and of relationship among these objects. Entities are described in a database by a set of attributes.

8.What is Object Oriented model?

This model is based on collection of objects. An object contains values stored in instance variables with in the object. An object also contains bodies of code that operate on the object. These bodies of code are called methods. Objects that contain same types of values and the same methods are grouped together into classes.

9.What is an Entity?

An entity is a thing or object of importance about which data must be captured.

10.What is DDL (Data Definition Language)?

A data base schema is specifies by a set of definitions expressed by a special language called DDL.

11.What is DML (Data Manipulation Language)?

This language that enable user to access or manipulate data as organized by appropriate data model. Procedural DML or Low level: DML requires a user to specify what data are needed and how to get those data. Non-Procedural DML or High level: DML requires a user to specify what data are needed without specifying how to get those data

12.What is DML Compiler?

It translates DML statements in a query language into low-level instruction that the query evaluation engine can understand.

13.What is Query evaluation engine?

It executes low-level instruction generated by compiler.

14.Under what conditions should indexes be used?

Indexes can be created to enforce uniqueness, to facilitate sorting, and to enable fast retrieval by column values. A good candidate for an index is a column that is frequently used with equal conditions in WHERE clauses.

15.What is difference between SQL and SQL SERVER?

SQL is a language that provides an interface to RDBMS, developed by IBM. SQL SERVER Is a RDBMS just like Oracle, DB2